"HelloWorld" Program in Java :-

// A java program to print Helloworld	Single line Command
Public class HelloWorld	Define a class
{	Open bracket for class
Public static void main(String[] args)	main method, entry Point of any java prgm
{	Open bracket for main
System.out.println("HelloWorld");	Output statement Prints "Helloworld" to console
}	close bracket for main
}	close bracket for class

How the above code runs?

1. Code is written in a file like "HelloWorld.java"

↓

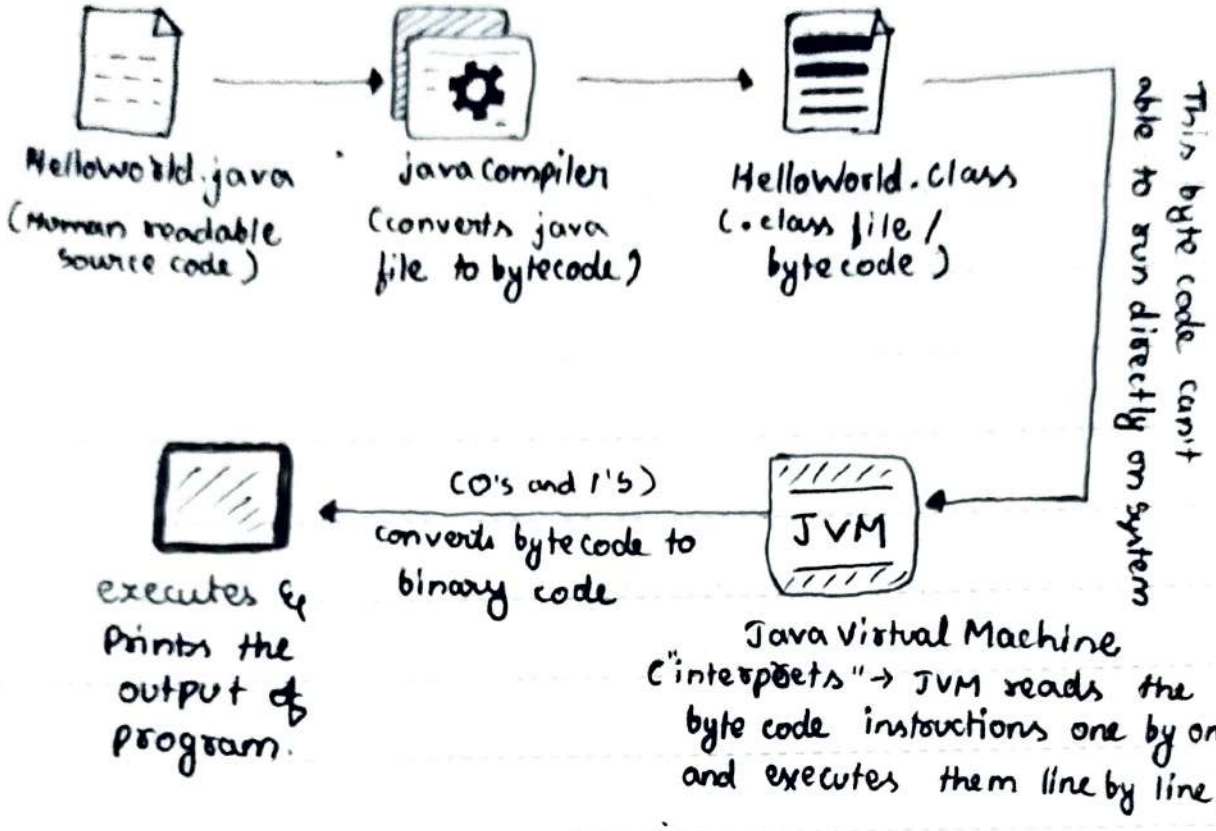
2. Java compiler "javac" compiles it into bytecode file "HelloWorld.class".

↓

3. JVM (Java Virtual Machine) reads .class filye and interpretes the bytecode

↓

4. JVM converts bytecode to Machine readable code i.e "binary" (0110..) and executes the program.



Why Java is Platform independence?

*) Once the .java file is compiled by compiler and byte code (.class file) is created.

*) This byte code can run on all OS.
"Any byte code witten in any software can run in any other software."

How byte code runs in all OS?

*) For example: If we compile c/c++ file on windows, we can't run it on mac OS. we need to compile it again in macos.

*) C/C++ provides machine code directly when compiled which is OS-specific and can't able to run same in other OS.

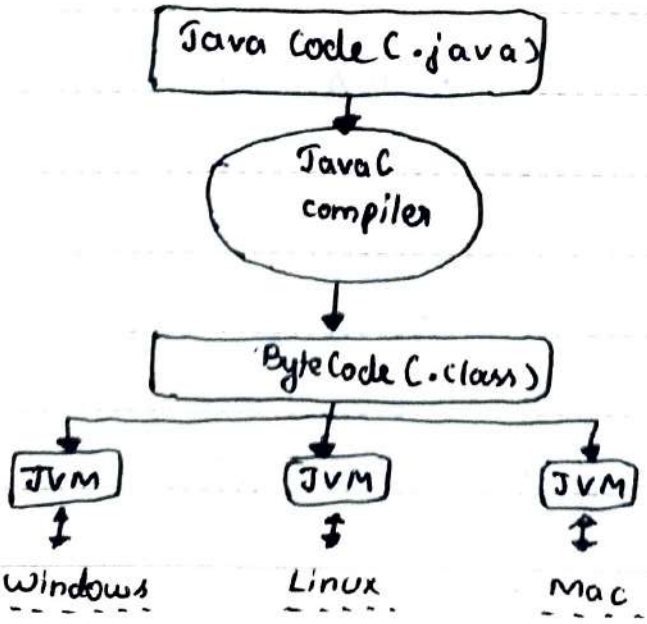
*) That's why, Java converts it into "bytecode" not "machine code"

*) And then JVM converts the "bytecode" to "machine code".

Each OS has its own JVM ("platform dependent")

- Windows → windows JVM
- Linux → Linux JVM
- Mac → Mac JVM

JVM understands byte code and converts it into native machine code.



Java has ability of "WORA"

Write Once
Run Anywhere

Architecture of Java :-

Think of Java execution like this :

You write code → JDK compiles → JRE runs
↓
JVM executes.

JDK → for building (developer)
JRE → for running (user)
JVM → actual execution engine

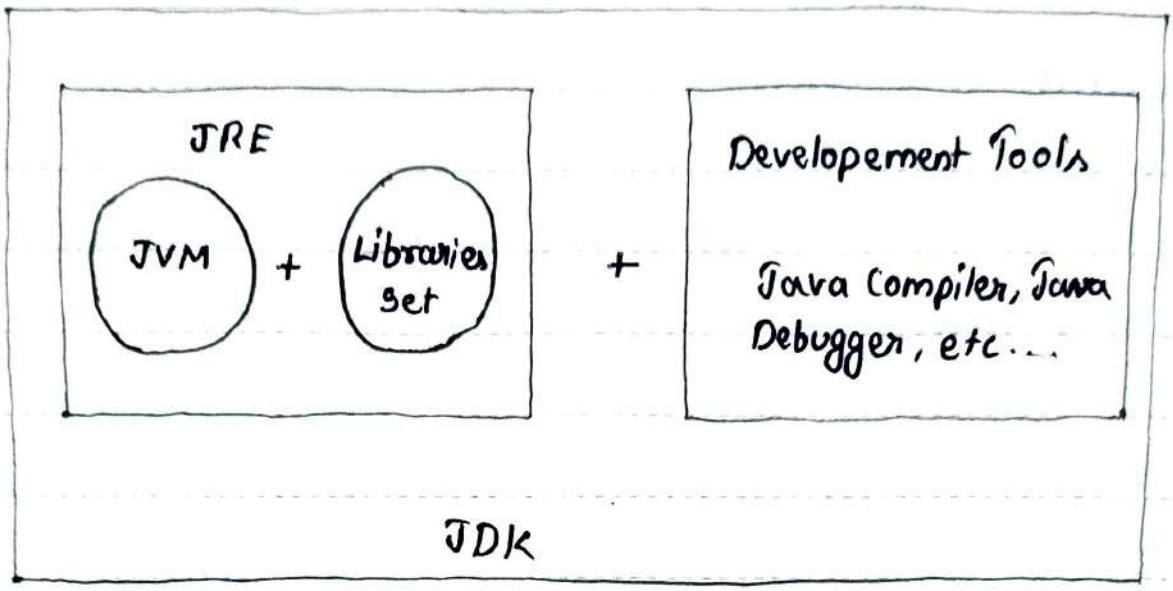
What is JDK ?
(Java Development kit)

*) JDK is a complete package for developers to create Java programs.

*) It provides tools and libraries to develop java applications, working with JRE & JVM

*) where as JRE → offers libraries and JVM, needed to run Java program.

JVM → executes the compiled Java byte code on the system.



*) It contains:

1. JRE (to run programs)
2. Tools to build Programs

*) Important tools inside JDK

1. javac → converts ".java" → ".class" (bytecode)
2. java → Runs the program
3. jar → Packages files
4. javadoc → Generates documentation
5. debugger → Helps find errors

*) JDK is only for development (not needed for running java program)

*) JDK is platform-dependent (different version for windows, linux, mac OS)

What is JRE ?
(Java Runtime Environment)

*) JRE is a Environment required to run java programs.

*) It does not include development tools. It is intended for end-users who only needs to execute applications.

*) It contains:

1. JVM (execution engine)
2. Libraries (predefined classes like strings, ArrayList, etc...)

*) It does not support compilation or debugging.

*) It is platform-dependent (different builds for different OS).

*) Work process :-

1. Loads ".class" file to memory
2. Verify the bytecode (Security)
3. Sends it to JVM for execution.

What is JVM?
(Java virtual Machine)

*) JVM is core component of JRE that allows Java program to run on any platform without modifications.

*) JVM is engine that actually runs your program, it converts bytecode → machine code.

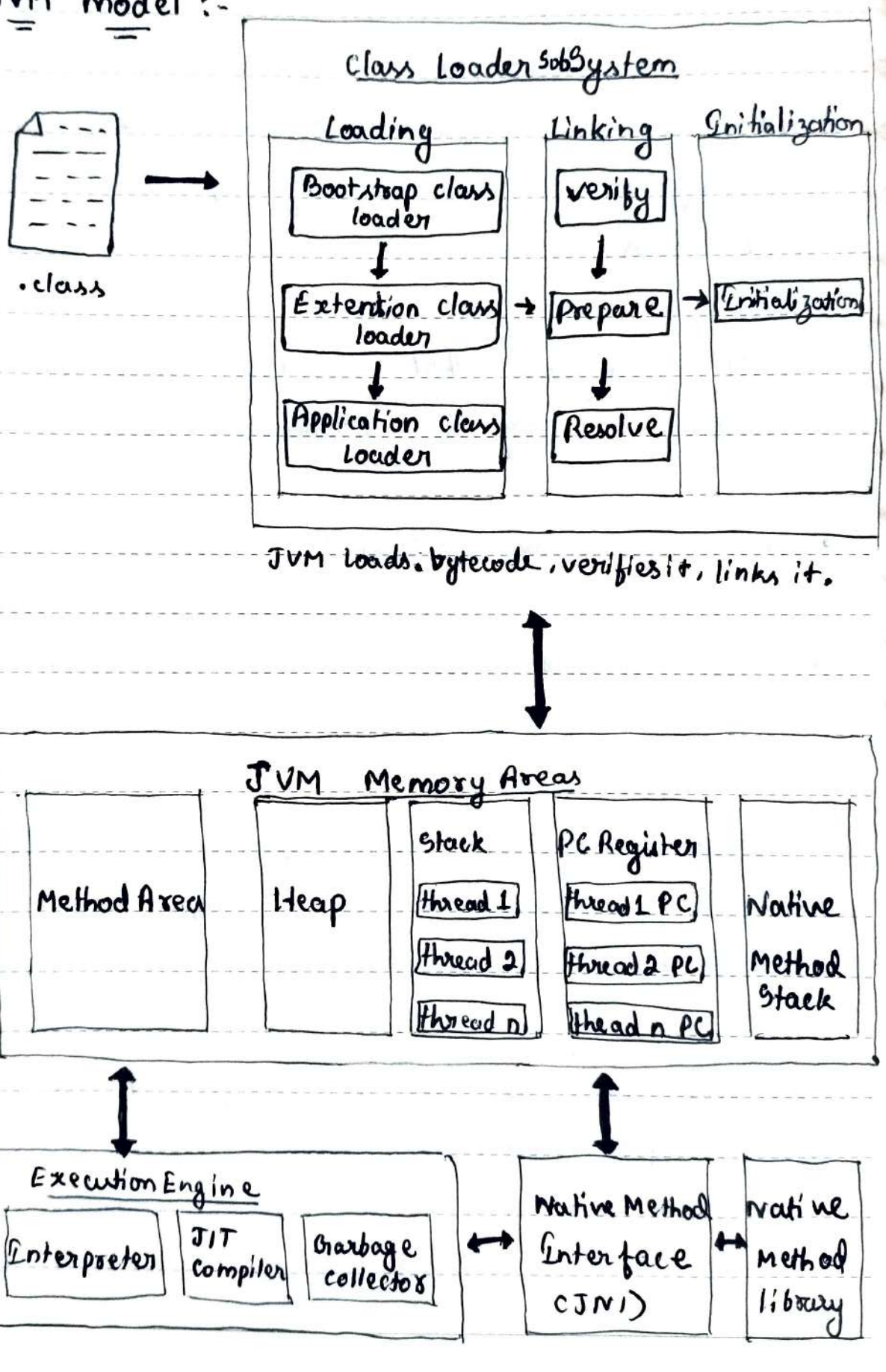
- Java source (.java) → compiled by javac → bytecode (.class)

- JVM loads the byte code, verifies it, links it, and then executes it.

- Execution may involve interpreting bytecode or using "Just-In-Time" (JIT) compilation to convert "hot" code into native machine code.

- Garbage collection runs in background to reclaim memory from unused objects.

JVM model :-


Class Loader SubSystem
.class
Loading
Bootstrap class loader
Extention class loader
Application class loader
Linking
verify
Prepare
Resolve
Initialization
Initialization
JVM loads bytecode, verifies it, links it.
JVM Memory Areas
Method Area
Heap
Stack
Thread 1
Thread 2
Thread n
PC Register
Thread 1 PC
Thread 2 PC
Thead n PC
Native Method Stack
Execution Engine
Interpreter
JIT Compiler
Garbage collector
Native Method Interface (JNI)
Native Method library

Components of JVM Architecture :-

1. Class Loader Subsystem :-

*) It is mainly responsible for three activities :

1. Loading :

→ Reads ".class" files & Stores class metadata in the method

→ It will create Class object in the heap representing the loaded class.

2. Linking :

→ It prepares the loaded class for execution.

→ Three Steps :

Verification	→	Preparation	→	Resolution
Ensures bytecode follows JVM rules and safe to execute		Allocates memory for static variables and assigns default values.		converts symbo-lic refrences into direct refrences in memory

3. Initialization :

→ Assigns actual values to static variables

→ It will execute static blocks defined in class.

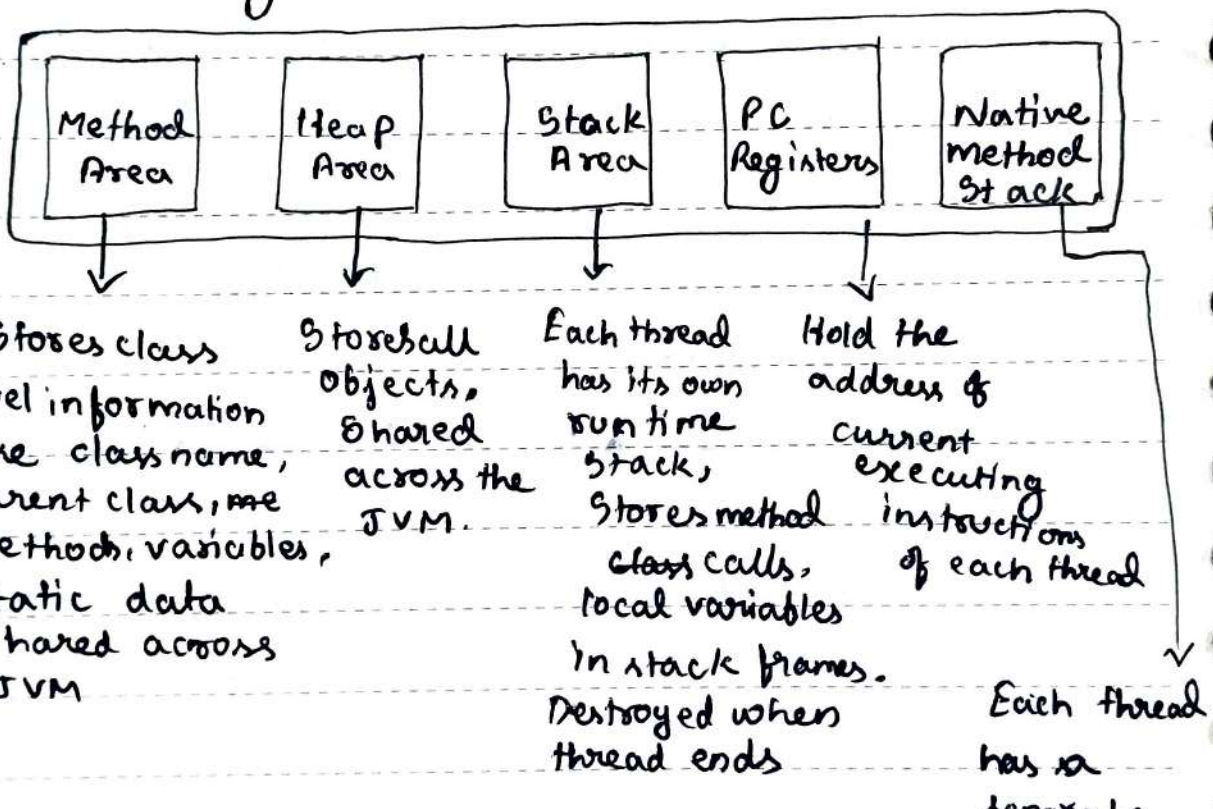
*) Class Loader types :

Bootstrap class loader → Loads core Java classes

Extension Class Loader → Loads classes from extensions directory

System/Application class loader → Loads classes from the application classpath.

2. JVM Memory Areas :



3. Execution Engine :-

*) Executes the ".class" (bytecode)

*) reads byte-code line by line

*) It uses data and info present in various memory area and executes instructions.

*) It is classified into 3 parts :

Interpreter → It interprets bytecode line by line, but when one method called multiple times, every time interpretation is required.

Just-In-Time Compiler (JIT) → Used to increase the efficiency of an Interpreter.

*) It compiles entire bytecode & changes it to native code so whenever interpreter sees repeated method calls; JIT provides native code for that part.

*) So, no need for interpretation again & again thus, efficiency increases.

Garbage Collectors → It destroys un-referenced objects.

4. Java Native Interface :-

*) It is an interface that interacts with the Native method Libraries and provides the native libraries (c, c++) required for execution.

*) It enables JVM to call C/C++ libraries and to be called by C/C++ libraries specific to hardware.

5. Native Method Libraries :-

*) These are the collections of native libraries required for executing native methods.

*) They include libraries written in C and C++.

Famous Applications built Using Java :-

- Netflix
- Amazon
- LinkedIn
- MineCraft
- Spotify
- Uber
- NASA World wind

What we can do with JAVA?

- Mobile App Development
- Web Development (using frameworks like Spring, SpringBoot, Struts, Hibernate)
- Desktop GUI Appalications (libraries - JavaFX & Swing)
- Enterprise Applications : banking, etc...
- Game Development : with engines LibGDX & jMonkey Engine
- Big Data Technologies : Hadoop & Apache Kafka
- Cloud-based Applications : AWS, Azure & GC where Java used in Services.